

阅读文献笔记

vLLM论文笔记

背景

- LLM采用**自回归**的算法，会对前文提到的东西产生依赖，只能够串行生成，无法并行生成。

自回归本质是条件概率的连乘。 $P(x)=P(x_1)P(x_2|x_1)P(x_3|x_1,x_2)*...$ ，用户的prompt阶段可并行，自回归生成阶段会有数据依赖。

- 自回归生成阶段，矩阵和向量相乘导致的GPU算力大量闲置，而prompt阶段是矩阵乘矩阵是可并行的。
- 用batching（批处理）把多个请求凑成一批送进模型计算，所以提高batch size就可以将memory-bound提高，提升吞吐量。
- Batch size上限被KV Cache显存卡住。

KV Cache是键值缓存，如果不缓存，生成每一个token都要重算前面所有的，所以同一个token出现在不同位置的KV不同。并且KVcache随生成动态增长，生命周期和长度都不知道。

- 低效问题，浪费。tensor（多维数组）必须连续存储，导致了必须为每个请求按照最大可能长度分配显存，造成了三种浪费。并且无法做到多序列场景（如beam search）的内存共享。
 - 预留槽位，对未来token预留空间。
 - 内部碎片，按最大长度，尾部经常会有空余用不到。
 - 外部碎片，各个请求分配大小不同，导致空洞（这块不太理解）

框架算子要求 KV cache 张量连续

- └> 现有系统把一个请求的 KV cache 存成【一整块连续 buffer】
 - └> 但输出长度事先不可知
 - └> 只能按【最大可能长度】预分配整块空间
 - └> 内部碎片、外部碎片、预留浪费（有效利用率仅 20.4%–38.2%）
 - └> 每个序列各自一块连续空间 → 无法跨序列共享

方法：PagedAttention+vLLM

3.2 PagedAttention 算法 (§4.1)

把每个序列的 KV cache 切成固定大小的 KV block (块大小 B)，块与块不必物理连续。标准 attention (Eq. 3) 被改写为分块计算 (Eq. 4)：

$$A_{ij} = \frac{\exp(q_i^\top K_j / \sqrt{d})}{\sum_{t=1}^{\lceil i/B \rceil} \exp(q_i^\top K_t \mathbf{1} / \sqrt{d})}, \quad o_i = \sum_{j=1}^{\lceil i/B \rceil} V_j A_{ij}^\top$$

其中 $K_j = (k_{(j-1)B+1}, \dots, k_{jB})$ 、 V_j 为第 j 个块。kernel 按块表**分别找到并读取**各块，先算 query 与块内 key 的分块 attention 分数 A_{ij} ，再与块内 value 相乘累加得到 o_i 。

这个公式让KV cache变为KV block，分块计算，让KV可以散到非连续的显存中。

- 利用KV Cache Manager让请求的KVCache表示为一串逻辑块，申请一大段DRAM切分为物理块，然后建立块表 (block table)进行映射，最后做到按需分配。
- 解码流程：比如7个token分配2个逻辑块，块0存储前四个，块1存后三个，映射到两个物理块7，1，剩一个空位就留给下一步，然后进行两步的自回归，第一步用PagedAttention读取物理块，然后新的token写入块1的空位，若块1满了则进行第二步，新分配一个物理块3作为新逻辑块，然后写映射。

"所有盘从左侧填满，只有当前块未满" $\Rightarrow$ 每个请求的浪费被限制在至多一个块内。

复杂解码场景

引用计数：当前有多少个逻辑块指向一个物理块

COW（块粒度写时复制）：写数据新前物理块将原块全部内容拷贝过去，不对原块进行修改。

fork、append、free：派生新序列，新序列逻辑块指向父序列的物理块。写新KV到最后一个块，块满则分配新物理块，ref>1则先触发COW（因为不止一个逻辑块指向这个物理块，不能对原块修改）。删除一个序列（逻辑块）

并行采样

- 场景：一个prompt，给多个候选答案让用户挑一个。

复制

只分配 2 个物理块：物理块 7 (存前 4 个 token)、物理块 1 (存后 3 个 token)

Sample A1: 逻辑块0 ─┐ 逻辑块1 ─┐
 └─> 物理块 7 └─> 物理块 1
Sample A2: 逻辑块0 ─┐ (ref=2) 逻辑块1 ─┐ (ref=2)

关键动作 = 一次 `fork` : A2 从 A1 派生, 逻辑块直接指向同一批物理块, 于是两个物理块的 `ref count` 都变成 2。

第2步：生成开始，两个 sample 分道扬镳

- A1 采样出 "fathers", A2 采样出 "mothers" → KV 不同, 不能再共享
- 两者都要往自己的最后一个逻辑块 (逻辑块 1) 追加 → 而逻辑块 1 对应的物理块 1 此时 `ref = 2`

- 在不同的采样过程中，应该要触发COW，因为A1, A2都想写入物理块，但此时的KV不同了，A1拷贝一份新的，此时ref=1，所以A2就不用拷贝了，直接用原物理块。
- 因此假如不共享的prompt，逻辑块占N个，需要2N个物理块，用了vLLM便只需要N+1个物理块，N越大节省越多。

Beam Search

不同于贪心以及穷举，是一个折中的做法。

1. 首先将得到的k个候选用可能的token扩展得到 $k \times |v|$ 个候选，然后用LLM算出来概率，保留概率最高的top-k个，丢弃其余。

拿这 3 条，各自用全部 6 个词扩展 → 得到 18 条临时候选，算概率（这里只列关键的）：

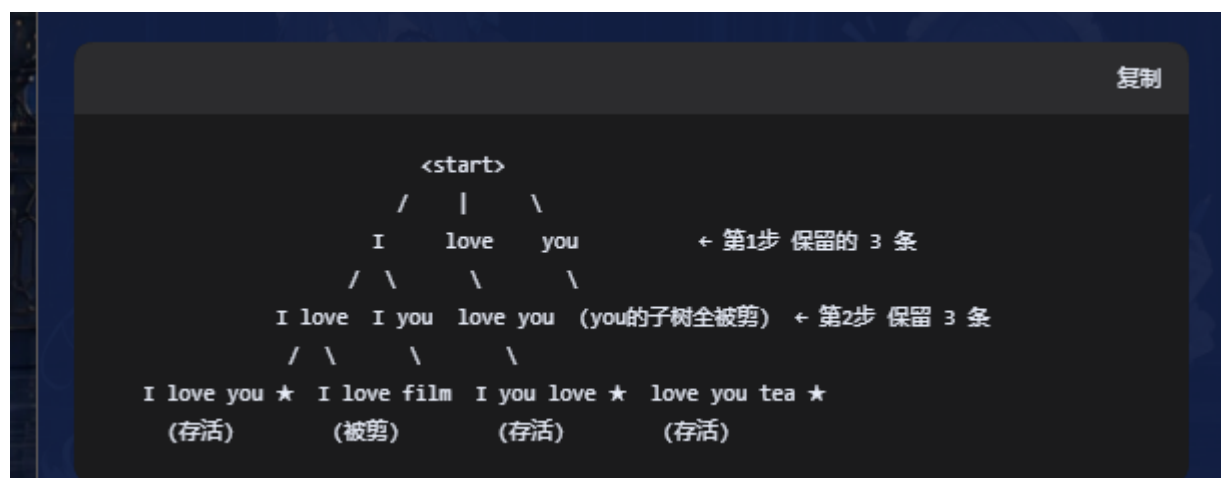
候选	计算	概率
I love	0.6×0.5	0.30 ✓
I you	0.6×0.2	0.12 ✓
love you	0.2×0.6	0.12 ✓
I tea	0.6×0.1	0.06 ✗
love I	0.2×0.28	0.056 ✗
you love	0.1×0.4	0.04 ✗
...		

保留 top-3 → I love (0.30)、I you (0.12)、love you (0.12)

🔊 注意这里：you 这条分支死了（它的后代概率太低，全部被剪掉）。而 I love 和 I you 共享前缀 I。

而且注意：这 3 条候选的祖先完全不同——前两条来自父亲 I，第三条来自父亲 love。

- 进行扩展打分，最后形成一棵树，在树中天然共享前缀，而树的每一步都在变，死的分支用free，活的继续fork。



共享前缀

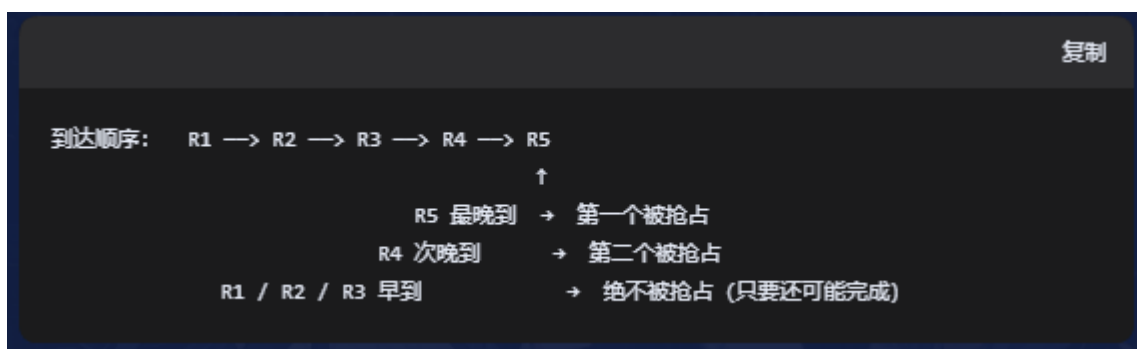
- 这个在SGLang写的过程中已经比较清楚了，在这里逻辑块和物理块的作用体现在，prompt的前缀部分逻辑块可以直接映射到已经缓存过的物理块，然后最后一块标记为COW进行拷贝进行接着书写。

混合解码

调度与抢占

LLM中，一个正在解码的请求一定要一直待在显存里，新token的生成要attention来访问全部历史的KV，块被挪走无法算，当新token需要新物理块，但池子不够装了，就要解决驱逐块和驱逐的块如何恢复两个问题。

- 调度策略：**FCFS（先来先服务）**，保证公平，这一块并非最优策略，而是公平性的选择，是一个价值取舍。



- 传统OS的驱逐策略是启发式预测，哪个页最久不会被访问才要换出去，但是在LLM中不适用，因为attention需要访问序列中从头到尾所有位置的KV，所以要么全驱逐，要么不驱逐，也就是all-or-nothing。
- Sequence group：序列组内的所有序列，永远一起被抢占，一起被重新调度。

假设 bundle 里有 A1、A2 两条序列，共享物理块 P (`ref = 2`)。现在内存紧张，我们只驱逐 A1 (不驱逐 A2)：

驱逐 A1: `P.ref: 2 → 1` ← 块 P 还在被 A2 用, 【不能回收】
结果: 一个物理块都没释放出来 ❌

只赶走一半 = 一点内存都收不回来 (因为共享部分仍被另一半占着)。

所以必须整组赶走：

驱逐整组 (A1 + A2) : `P.ref: 2 → 0` → 块 P 立即回收 ✅

而且如果只驱逐一半，恢复时还会遇到“共享关系断裂”的一致性噩梦。

🔗 一句话：共享内存的存在，使得驱逐必须与共享边界对齐。

- 恢复被驱逐的块

1. Swap：空间换时间，将被驱逐的块拷贝到CPU保存，有需要再拷贝回来
2. Recomputation：被抢占后把KV直接丢掉，重新调度时候重新算一遍，由于重算是“批量计算”，是矩阵乘矩阵的并行计算，所以很快，不过吃GPU算力和时间，但是省内存空间。

分布式执行

- 采用张量并行TP，Megatron-LM风格。

将一个大权重矩阵W，按行或按列切成几块，每张卡拿一块，各自算结果，也就是每个worker只计算自己的attention互不打扰。

- All-reduce，由于N张卡各有一个张量，操作结束后每张卡都拿到所有卡的张量之和。
- 单一集中式KV Cache Manager，逻辑位置的映射在所有worker上一致，所以所有worker共享同一份“逻辑块->物理块”的映射。

	每 worker 一个 manager	单一集中式 manager
映射一致性	各自分配 → 映射可能不一致 → 需要分布式共识/加锁	天然一致 ✓
抢占决策 (\$4.5)	所有 worker 必须协调着一起驱逐	scheduler 一人说了算 ✓
ref count (\$4.4)	分布式引用计数, 极难维护	单点维护, 简单 ✓
通信开销	每次分配都要同步	每步广播一次小块表即可 ✓

SGLang论文笔记

背景

- LLM 的使用正从"简单聊天"走向"用程序调度和控制生成过程", 这类程序称为 LM Program [4, 20]。它具有两个共性：
 - 包含多个 LLM 调用, 并与控制流交织 —— 为完成复杂任务、提升质量；
 - 接收结构化输入、产生结构化输出 —— 为便于程序组合、集成进既有软件系统。
- 挑战：
 - LLM非确定性导致开发困难, 大量的字符串拼接, 提示词实验性调参, 多模态的输入处理。
 - 执行低效, KV Cache复用机会被浪费程序内多个 LLM 调用共享公共前缀时, 本可复用 KV cache; 但现有系统在请求处理完后直接丢弃 KV cache, 导致重复计算与内存浪费。约束解码逐 token 进行: 结构化输出 (如 JSON) 用正则约束时, 某些 token 序列唯一确定、本可一次解码多个 token, 但现有系统每次只解码一个 token。

RadixAttention——KV缓存复用

- KV cache计算只依赖于前缀token, 因此共享相同提示前缀的请求可以复用
- 设计: 将KV cache当做一棵基数树——radix tree来管理, 保留缓存, 让缓存和运行中的请求共享同一个内存池没动态分配。系统用它来管理token序列, 也就是KV cache张量的映射, 以非连续分页布局来存储。

- 运用LRU驱逐策略：
 1. GPU显存很快被填满，则优先驱逐最近最少使用的叶子
 2. 先驱逐叶子使得公共祖先可以被复用。
 3. 在连续批处理中，不能驱逐正在运行批次使用的节点，一定要在引用计数器为0才可驱逐。
 4. 等待请求够多，系统驱逐所有缓存token来换取batch size。
- 缓存感知调度
 1. 缓存命中率：等待队列中请求的执行顺序会影响命中率，无关请求内频繁切换会导致缓存抖动。解决方案：按照匹配前缀长度排序，优先执行匹配前缀更长的请求。
 2. 按照深度优先搜索顺序访问请求的基数树可达到最优缓存命中率。

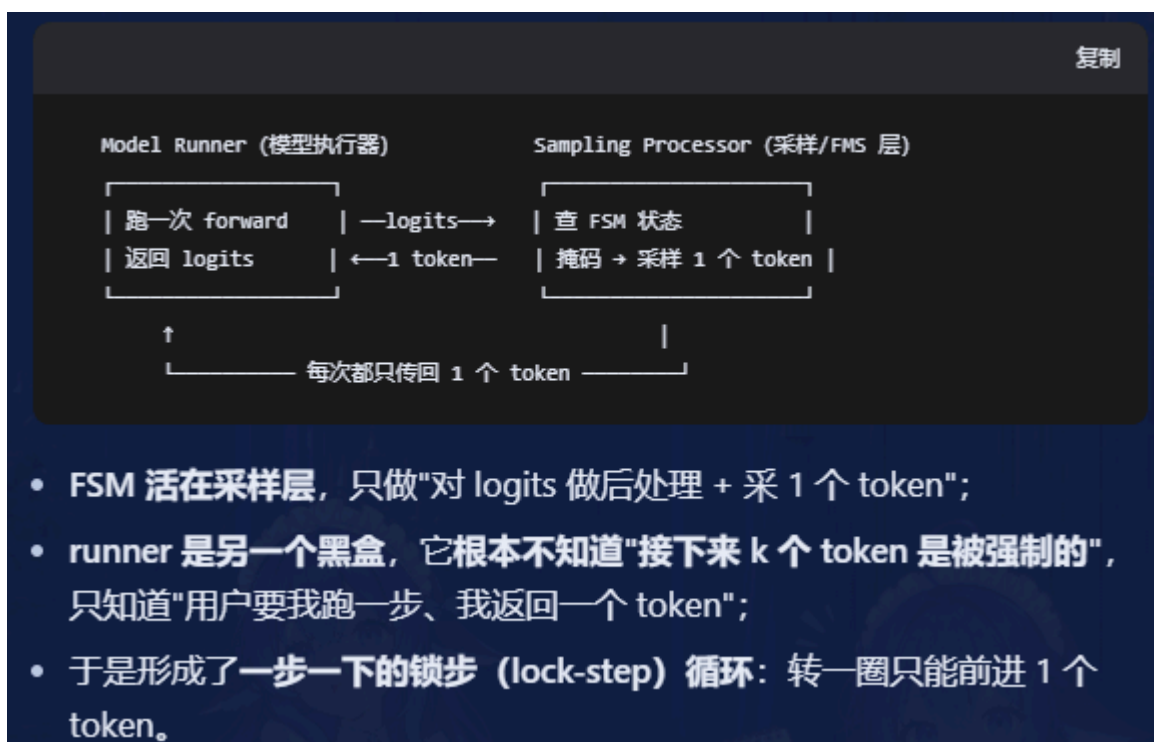
证明思路 (附录 A.3)： 设 C 为总计算复杂度，每条边 e 的 KV 至少被计算一次，故下界 $C \geq \sum_{e \in \text{edges}(T)} |e|$ 。按 DFS 访问时，首次计算边 e 后会连续命中整棵子树直至完成，且缓存足够大 ($\geq$ 最长路径) 使 e 在子树计算期间不被驱逐，因此每条边恰好计算一次，达到下界，命中率最优。随后用归纳法证明 longest-shared-prefix-first 等价于 DFS。在线场景下 DFS 顺序会被打断，但该调度仍近似隐式基数树扩展部分上的 DFS 行为。

DFS:遇到岔路，先选一条一直往里走，走到死胡同就原路退回到最近的那个岔路口，再试下一条。

压缩有限状态机

问题

- SGLang用正则表达式来表达一个“合法集合”，在采样之时将不合法的token概率压成0。
- 传统机制中，正则表达式就是将正则转成FSM（有限状态机），变成节点和边的一个图（自我理解上应该和离散数学里面的差不多），然后再用FSM指导掩码，将非法token的logit概率打成9，生成的字符串就100%在正则语言的字符串里。
- 但是这样子会低效，模型没有自由，后面的字符是被行文强行规定的，一步步让模型生成下一个词的意义就是为了生成KV cache，既然已知token，那就可以像prefill一样一次性全部喂进去，但是模型都做不到，比如 `{"summary": " "}` 这个语句用tokenizer要进行四次前向，每个位置的合法token集合都确定了，4次纯属浪费。
- FSM一直在采样层，只做“对logits做后处理+采1个token”，runner是“用户让我跑一步我跑一个token”。便导致了转一圈只能前进一个token。



方案：压缩FSM

- SGLang提出方案是协同设计，让FSM的信息穿透到runner，告诉它“这段不用问模型，直接跳过。”

1. 奇异转移边：

它的源节点只有一个后继节点（没有分支）

这条边上只有一个可接受的字符/字符串（没有选择）

说人话就是没得选了，只能往后。

- 2. 压缩边：由连续相邻的边($e_0, e_1, \dots, e_k$) 压缩而成，当且仅当 $e_1, \dots, e_k$ 都是奇异转移边。压缩边的文本 = 各边文本的拼接。

压缩算法：递归合并

从基于字符的 FSM 出发，递归地把奇异转移边合并进它前面的边，直到再也压不动为止。

用 `{"summary": " "}` 举例：因为每一个节点都只有一个后继、每条边都只有一个字符，所以整条链都是奇异转移边。

Jump Forward（跳跃式解码）

- 如果当前状态的出边是一条压缩边 → 说明后面这一整段文本是被强制的；
- 于是直接把压缩边上的整段文本转成 token 序列，一次性（作为 mini-prefill）喂给模型；
- 一次前向就完成了原本需要 k 次前向的工作，这就是论文说的 "Jump Forward"。

这一点很重要，容易和“投机解码（speculative decoding）”混淆：

	压缩 FSM	投机解码
本质	确定性跳过（这一段本来就没有选择）	猜测 + 验证（有猜错的可能）
是否改变结果	不改变（输出的文本、token 都被强制）	需要 accept/reject 校正
类比	直接快进已知剧情	押注下一段，错了回退

因为跳过的这段**本来就没有自由度**（掩码后唯一 token 概率=1），所以压缩 FSM 是**无损的精确优化**，而非近似。论文也据此把它算在“不改变计算结果”的优化里。

API投机执行

问题

- 对**只能通过黑盒 API 访问**的模型（如 GPT-4），我们无法修改推理过程。
- 多调用程序会产生**重复的输入 token 费用与延迟**。例：

```
python
```

复制

```
s += context + "name:" + gen("name", stop="\n") + "job:" + gen("job", stop="\n")
```

两次 `gen` = 两次 API 调用 → **context 的输入 token 费用被付了两次。**

方案

- 在第一次调用时启用投机执行：忽略 stop 条件，让模型多生成几个 token。
- 解释器保留这些额外生成结果，在后续原语中匹配并复用。
- 在精心设计提示词的情况下，模型能以高准确率匹配模板，从而省下一次 API 调用的延迟和输入成本。